

PORTFOLIO

INTRODUCE

오준석

BackEnd Developer

Contact.

Email. work.ditto909@passmail.net

Phone. 010-6657-2531

Channel.

Resume. junseokhub.github.io

GitHub. github.com/junseokhub

Blog. junseokoo.tistory.com/

Experience

Gentlepie

2022.04 ~ 2025.05 (BackEnd Development)

PROJECTS

Bublitt

2023.09 ~ 2025.05 | Spring Boot, Milvus, Kubernetes, ArgoCD

프로젝트 배경

다양한 고객사의 챗봇을 하나의 플랫폼에서 관리하는 AI 챗봇 빌더. 초기 NestJS 기반 서비스를 Spring Boot로 전환하며 AI 응답 품질, 동시성, 비동기 처리까지 전반적인 시스템 고도화를 주도.

성과

- RAG 아키텍처 + ReRank 모델 도입으로 응답 정답률 **65.24% → 93.71%**
- DBSCAN 클러스터링 기반 주제 추천 시스템 개발로 **Fallback 발생률 20% 감소**
- Jenkins CI → ArgoCD GitOps 전환으로 배포 실패율 30% 감소
- AWS에서 온프레미스 Kubernetes로 전환해 인프라 운영 비용 약 50% 절감

기술적 의사결정

- ◆ RAG 응답 정답률 개선

[문제]

단순 벡터 유사도 검색만으로는 의미적으로 유사하지만 실제로는 관련 없는 문서가 상위에 노출

[원인]

Embedding 모델 한계 + 데이터 전처리 미흡으로 벡터 공간에 노이즈 혼재

[해결]

데이터 전처리 개선 + ReRank 모델 도입으로 검색 결과 재정렬 → 정답률 65.24% → 93.71%

- ◆ NestJS → Spring Boot 마이그레이션 동시성 문제

[문제]

싱글스레드(NestJS) → 멀티스레드(Spring Boot) 전환 후 토큰 과금, 벡터 DB 업데이트 등 동시 요청 시 정합성 깨짐

[원인]

NestJS는 싱글스레드 이벤트루프라 애플리케이션 레벨 충돌이 없었으나, Spring Boot 멀티스레드에서는 DB 레벨 충돌 발생

[해결]

충돌 특성에 따라 **낙관적 락(민감정보 수정) + 원자적 UPDATE(토큰 누적)**를 구분 적용 + 재시도 로직으로 정합성 확보

- ◆ 토큰 누적 처리량 개선 — 비관적 락 → 원자적 UPDATE

[문제]

토큰 누적을 비관적 락으로 처리했으나, 단순 증분 연산임에도 read-modify-write 구간을 통째로 잠가 핫로우에서 처리량을 제한

[해결]

행 단위 **원자적 UPDATE**로 전환해 애플리케이션 레벨 락 경합 제거. 락을 DB 내부 row latch에 위임

[향후]

핫로우에서 p99 지연 상승 시 Redis 카운터(단순 누적은 INCRBY, 조건부는 Lua) → 정합성은 Outbox → 단일 키 병목 시 카운터 샤딩 → Kafka 집계 순으로 에스컬레이션 기준 정의

- ◆ I/O 블로킹 문제

[문제]

OpenAI Embedding → Milvus 검색 → OpenAI Chat Completion 3개의 외부 API 순차 호출로 서블릿 스레드가 응답 대기 동안 점유되어 처리량 저하

[원인]

RAG 파이프라인 특성상 3단계가 순차 의존 관계라 병렬화 불가능

[해결]

WebFlux 전환은 MVC 레거시 전환 비용이 효과 대비 높다고 판단, **CompletableFuture + 전용 ioExecutor 분리**로 서블릿 스레드 점유 방지 + Timeout 처리

⚠ 한계 및 향후 방향

- DBSCAN eps 파라미터 불안정(매주 데이터 밀도 변동) → **HDBSCAN** 도입 검토
- 비동기 후처리(토큰·로그 적재)는 CompletableFuture의 thenAcceptAsync로 채팅 응답과 분리 중
→ 처리 대상 확대 시 **Kafka 이벤트 드리븐** 전환 검토

✿ 마무리

단순히 동작하는 시스템을 넘어 *왜 이 구조인지, 어떤 한계가 있는지를* 끝까지 파고들며 기술적 의사결정과 트레이드오프를 고민하는 습관을 만든 프로젝트

식약처 코스봇

2024.05 ~ 2025.05 | Spring Boot, Milvus, Kubernetes

🔍 프로젝트 배경

기존 분류형 챗봇은 정보 탐색 난이도가 높고 기계적인 말투로 만족도가 낮았다. Bublitt 플랫폼 기반 Upstage RAG(Solar)를 도입해 식약처 맞춤형 LLM 챗봇으로 전환

📊 성과

- Embedding 모델, 벡터 차원별(256/512/1024/3072/4096) 응답 정확도 자동 비교 테스트 모듈 개발로 최적 모델 선정 체계 구축
- 기관 맞춤형 통계 대시보드로 정책 의사결정 데이터 시각화 제공
- Prometheus + Grafana 모니터링 + ArgoCD GitOps로 운영 안정성 확보

⚙️ 기술적 의사결정

- ◆ Embedding 모델 선정 체계 구축

[문제]

모델마다 응답 정확도가 다르고 벡터 차원에 따라 검색 성능이 달라, 어떤 조합이 최적인지 기준이 없었음

[해결]

Upstage·OpenAI 등 Embedding 모델별 + 벡터 차원별 응답 정확도를 자동 비교하는 테스트 모듈 개발
→ 정답률·응답 속도·비용을 종합 평가해 데이터 기반으로 최적 모델을 선정하는 검증 체계 확립

- ◆ 기관 특화 통계 시스템

[문제]

일반적인 챗봇 통계로는 식약처 특성에 맞는 인사이트 도출이 어려움

[해결]

식약처 업무 패턴에 맞춘 통계 로직 설계 + 대시보드 개발로 정책 의사결정 데이터 제공

⚠️ 한계 및 향후 방향

핵심 RAG 로직이 Bublitt 플랫폼에 종속되어 식약처 특화 커스터마이징에 한계 → 도메인 특화 Fine-tuning 또는 독립 RAG 파이프라인 구축 검토

✿ 마무리

플랫폼 연동 프로젝트에서도 기관 맞춤형 검증 체계와 통계 시스템을 직접 설계하며, 도메인 이해가 기술 구현만큼 중요하다는 것을 체감한 프로젝트

Coconutive

2023.10 ~ 2025.05 | Nest.js, Redis, MySQL, MongoDB

🔍 프로젝트 배경

챗봇 사용 데이터를 수집·분석하는 통계 솔루션. Raw SQL 기반 레거시 코드를 점진적으로 개선하며 운영 안정성과 데이터 관리 체계를 구축

📊 성과

- Raw SQL → TypeORM 전환(70%+ 완료)으로 타입 안전성 확보 및 유지보수성 향상
- Redis 중앙 집중형 세션 관리로 **중복 로그인 32% → 0%** 해결
- 자연어 기반 통계 → LLM 기반 통계로 전환해 분류 자동화 및 정확도 향상
- 수동 로그 관리 → MongoDB CronJob 자동화로 운영 안정성 확보

⚙️ 기술적 의사결정

- ♦ Raw SQL → TypeORM 전환 전략

[문제]

고객사별 DB 환경이 달라 LIMIT/OFFSET·날짜·JOIN 문법이 조금씩 달라 DB별 쿼리 유지보수 부담 증가

[원인]

초기 Raw SQL로 빠르게 개발하다 보니 DB 종속적인 쿼리가 전체에 혼재

[해결]

100% 전환을 시도했으나 다중 JOIN·동적 필터·집계 함수 등 복잡한 쿼리는 TypeORM이 오히려 가독성이 떨어진다고 판단 → **단순 CRUD/동적 필터는 TypeORM, 복잡한 통계/집계는 Raw SQL 유지**로 기준을 명확히 정의해 70%+ 전환

- ♦ Redis 세션 관리 도입

[문제]

중복 로그인 방지가 필요했으나 기존엔 검증 로직이 없어 동일 계정 다중 접속 허용

[해결]

로그인 시 Redis에서 기존 세션 존재 여부 확인 후 만료 처리 + 신규 세션 발급 → 중복 로그인 32% → 0%

⚠️ 한계 및 향후 방향

복잡한 통계 쿼리는 Raw SQL이 가독성·성능 면에서 현실적으로 우위 → 남은 Raw SQL은 DB별 분리 + Executor 패턴으로 런타임에 DB 종류 확인 후 로딩하는 방향 검토

🌱 마무리

모든 코드를 한 번에 바꾸는 것보다 비즈니스 연속성을 유지하며 점진적으로 개선하는 것이 엔지니어링의 본질을 체감한 프로젝트

SK-EcoPlant 사내 챗봇

2023.04 ~ 2025.05 | Nest.js, TypeORM, DialogFlow CX, AWS

프로젝트 배경

내 포털의 직원 불편접수가 SNS로 퍼지며 운영팀 피로도가 높아지는 문제를 해결하기 위해, 대화형이 아닌 정보형 챗봇으로 접수 채널을 전환

성과

- DialogFlow CX Slot Filling으로 시나리오별 정보 자동 수집 및 문맥 기반 응답 구현
- 어드민에서 Intent/Entity 등록·수정·삭제 가능한 관리 API로 운영팀 자체 관리 체계 구축
- AWS S3 + CloudFront 기반 챗봇 SDK 배포 자동화
- Jenkins CI/CD + Load Balancer 기반 운영 환경 구축

기술적 의사결정

- ◆ 대화형 vs 정보형 챗봇 선택

[문제]

직원들이 불편사항을 SNS에 게시하는 문화가 형성되어 운영팀 피로도 증가 및 부정적 분위기 확산

[의사결정]

자유로운 대화형 챗봇은 오히려 불만을 증폭시킬 수 있다고 판단, 정해진 시나리오로 접수를 유도하는 정보형 챗봇으로 방향 결정

- ◆ 스프레드시트 실시간 동기화 문제

[문제]

접수 데이터를 구글 스프레드시트에 실시간 반영했으나, 담당자 실수로 행/열 삭제 시 복구 불가

[해결 과정]

폴링 방식은 리소스 낭비 확인 → 사용자 편의성보다 데이터 정확도,보안이 우선이라 판단해 어드민 페이지 구축으로 전환

한계 및 향후 방향

외부 도구(스프레드시트)에 의존하는 구조는 사용자 실수로 인한 데이터 손실 위험이 상존 → 초기부터 어드민 페이지로 설계했다면 더 안정적이었을 것이라는 교훈

마무리

기술적 구현보다 비즈니스 문제를 먼저 정의하고, 도구 선택이 운영 안정성에 미치는 영향을 직접 체감한 프로젝트.

운영 업무

신세계 면세점 상담톡 / 삼성닷컴 챗봇 (운영및 기능 개발)

2022.04 ~ 2023.08 | Vue.js, Express.js, OracleDB, DialogFlow ES, IBM Watson

🔍 프로젝트 배경

입사 초기 실서비스 운영을 담당하며 레거시 코드 개선과 신규 기능 개발을 병행
빠른 이슈 대응과 코드 품질 개선을 통해 서비스 안정성 확보

📊 성과

- 3개월 내 120건 이상 이슈 신속 처리로 서비스 안정성 확보
- 중복 함수·조건문 제거 및 모듈화로 레거시 코드 리팩토링, 신규 기능 추가 비용 절감
- AWS Cognito MFA 적용으로 어드민 2단계 인증 보안 강화
- Docker 배포 시 환경변수 주입 방식 적용으로 보안 취약점 해결

⚙️ 기술적 의사결정

중복 함수, 조건문 과다로 사이드 이펙트 파악이 어려운 상태

→ 한 번에 바꾸는 대신 기능 단위로 중복 제거 및 모듈화 → 서비스 중단 없이 점진적 개선

🌱 마무리

실서비스 운영을 처음 경험하며 빠른 이슈 대응과 레거시 개선을 병행하는 것이 얼마나 중요한지 체감한 프로젝트이며, 서비스를 멈추지 않으면서 개선하는 습관을 이때부터 만들었음

- 삼성닷컴 2024.12.31부로 서비스 종료

PERSONAL PROJECTS

퇴사 후 시스템 설계·성능 검증 중심으로 진행한 기술 심화 프로젝트. 상세 구현·실험 과정은 모두 블로그에 기록.

선착순 쿠폰·주문 동시성 처리 (전략별 부하 테스트 비교)

선착순 쿠폰/주문 재고 차감의 동시성을 **낙관적 락·비관적 락·Redis Lua·Kafka Async** 4가지로 구현하고 k6로 부하 테스트 비교. Kafka 심화(파티션·DLQ), Redis-DB 정합성(Outbox Pattern), WebFlux vs MVC + Virtual Thread 비교까지 진행

- 직접 처리는 WebFlux 우세, Kafka 비동기 발행은 MVC + Virtual Thread가 **15,685 TPS**로 압도
- 단일 기술의 우열이 아니라 **처리량/정합성/코드 복잡도의 트레이드오프**로 판단. 측정 환경의 한계(GC·네트워크)를 인지해 수치를 상대 비교로 해석

mini-broker (Kafka 브로커 직접 구현)

외부 프레임워크 없이 Kafka 브로커의 핵심(append-only log, sparse index, NIO TCP 서버, 파티션, 컨슈머 그룹)을 Java 21로 직접 구현

- JMH 벤치마크에서 첫 측정의 오류(비동기 vs 동기 혼동)를 직감으로 잡아내 공정한 비교 조건으로 수정
- 숫자보다 왜 그 숫자가 나왔는지를 이해하는 것이 벤치마크의 본질임을 체득

Kubernetes 하이브리드 클러스터

홈 서버(x86) + OCI ARM 클라우드 2개를 Tailscale로 묶은 3노드 Kubernetes를 운영

- x86/ARM 혼합 환경 라우팅, BuildKit rootless DNS, ArgoCD Image Updater 등 실운영 트러블슈팅 경험